

Appendix B: Biofilm Growth and Redistribution Algorithm

CompLaB3D models biofilm evolution through a cellular automaton (CA) that executes once per simulation time step. After reactions update the local biomass density, the CA checks whether any biomass voxel has accumulated more biomass than it can hold (Jung et al. 2023). If so, the excess is redistributed to neighboring voxels. This redistribution repeats in a loop until every voxel in the domain is within its capacity. Once redistribution is complete, the solver reclassifies voxels: biofilm voxels that have lost too much biomass become pore, and pore voxels that have gained enough biomass become biofilm. If any voxel changed classification, the pore geometry has changed, and the steady-state flow field is recomputed before advancing to the next time step. Planktonic (free-living) biomass in pore voxels is transported by advection and diffusion through the D3Q7 lattice and is not subject to the CA redistribution; only sessile biomass attached to the biofilm is redistributed by this algorithm.

Two user-defined parameters control the CA:

1. Maximum biomass density (`max_bMassRho` in the XML configuration, in $\text{kg}_{\text{DW}} \text{m}^{-3}$, where DW denotes dry weight). This sets the upper limit of biomass a single voxel can hold. Whenever the total biomass in a voxel exceeds this value, the surplus is redistributed.
2. Biofilm fraction threshold (`thrd_bFilmFrac`, a value between 0 and 1). This parameter represents the fraction of the maximum biomass density and defines when a voxel switches identity between pore and biofilm. Hence, a voxel is recognized as biofilm well before it becomes full enough to trigger redistribution.

B.1 The Main CA Loop

At each time step, after reaction kinetics have updated biomass densities, the CA finds the single highest biomass value in the entire domain (`globalBmax`). If that value exceeds `max_bMassRho`, the redistribution loop begins. Figure B1 shows a representative cross-sectional slice of the 3D domain before redistribution, with voxel types color-coded and Manhattan distances to the nearest pore labeled inside each cell.

— Algorithm B.1: Main Redistribution Loop —

Given: biomass density at every voxel, already updated by reaction kinetics

Goal: adjust densities so that no voxel exceeds `max_bMassRho`

1. Add up all species at each voxel to get the total biomass there. // one number per voxel
2. Label each voxel as pore, biofilm, or solid based on its current state.
3. Find the single highest total-biomass value across the entire domain. // call it `globalBmax`
4. Repeat while `globalBmax` is larger than `max_bMassRho`: // at least one voxel is overfilled
 - a. If the user chose full-push mode (`halfFlag = 0`):
Run the FULL-PUSH procedure (Algorithm B.2). // moves only the excess

```

Otherwise:
    Run the HALF-PUSH procedure (Algorithm B.3).    // moves half the total
b. Exchange biomass across MPI process boundaries (Section B.4).    // parallel sync
c. Recompute total biomass at every voxel.    // some values changed
d. Re-label voxel identities (pore/biofilm/solid).
e. Find the new globalBmax.    // check if another pass is needed
    // Safety: if loop exceeds certain number of iterations, terminate with error.
5. When the loop ends, every voxel is within capacity.
6. Run VOXEL RECLASSIFICATION (Algorithm B.5).    // update pore/biofilm labels

```

— End of Algorithm B.1 —

The user selects the redistribution mode with a configuration flag (`halfflag`). Both modes share the same two-stage neighbor selection described below and differ only in how much biomass is moved per iteration.

(a) Initial State — Before Redistribution

z-slice view | Numbers = Manhattan distance to nearest pore voxel

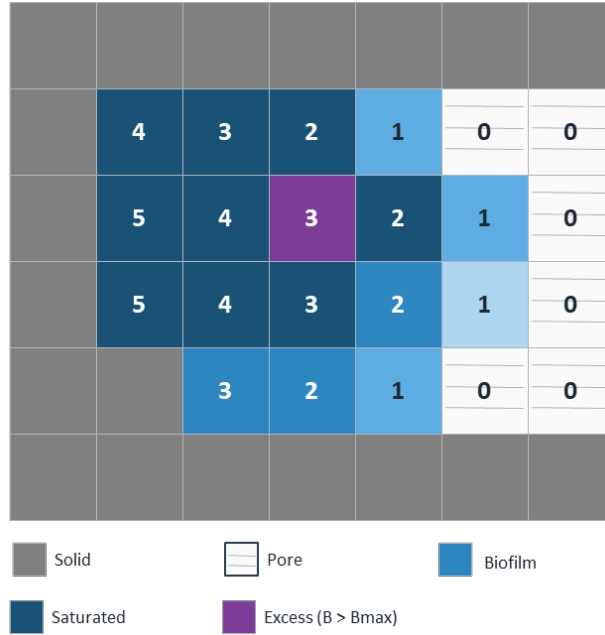


Figure B1. Initial state of a two-dimensional cross-section (*z-slice*) through the 3D biofilm domain before redistribution. Gray cells represent solid grains; white hatched cells represent pore space; shades of blue indicate biofilm voxels at increasing biomass density (darker blue = closer to saturation); and the purple cell shows a voxel whose total biomass exceeds the maximum allowed density B_{max} . The number inside each biofilm or pore cell is its Manhattan distance to the nearest pore voxel.

B.2 Full-Push Redistribution

The full-push scans every voxel in the domain. For each voxel whose total biomass exceeds `maxbMassRho`, the solver computes the excess and attempts to place it into neighboring voxels. Each interior voxel has six face-connected neighbors (left, right, front, back, top, bottom); voxels at domain faces, edges, or corners have fewer. Solid grains and closed (no-flux) domain boundaries are excluded from receiving biomass. At open (inflow/outflow) boundaries, voxels are treated as valid neighbors; excess biomass that reaches an open boundary face is removed from the domain. The redistribution proceeds in two stages, illustrated step by step in Figures B2–B5.

Stage 1: capacity-based placement (Figure B2 and Figure B3). The excess voxel and its face neighbors are first identified (Figure B2), along with each neighbor’s available holding capacity ($B_{\text{max}} - B$). The valid neighbors are then visited in random order (Figure B3). For each neighbor that is a biofilm voxel and has not yet reached the maximum biomass density, the model transfers as much of the excess as will fit. If the entire excess fits into one neighbor, the algorithm is done with this voxel and moves on. Otherwise, it fills that neighbor to capacity and continues trying the next neighbor. When multiple microbial species coexist at the source voxel, the transferred biomass is split proportionally so that each species contributes its share of the total.

— Algorithm B.2: Full-Push Redistribution —

Given: all voxel biomass densities

Goal: for each overfilled voxel, push the excess (amount above `max_bMassRho`) to neighbors

For each voxel `V` in the domain:

```
Skip V if it is a solid grain or a domain boundary. // biomass only lives in pore/biofilm
Skip V if its total biomass is within the limit. // nothing to redistribute here
```

```
Compute the excess: excess = totalBiomass(V) - max_bMassRho. // how much must leave
```

```
Collect the face-connected neighbors of V (up to 6 in 3D). // left, right, front, back, top, bottom
Discard any neighbor that is a solid grain or domain boundary. // biomass cannot go into walls
Shuffle the remaining neighbors into a random order. // avoids directional growth bias
```

```
-- Stage 1: try to fit the excess into biofilm neighbors --
```

```
Go through each neighbor N in random order:
```

```
Skip N if it is a pore voxel. // Stage 1 only sends biomass into existing biofilm
Skip N if its biomass is already at or above max_bMassRho. // no room left
```

```
Compute how much room N has: room = max_bMassRho - totalBiomass(N).
```

```
Decide how much to send: transfer = the smaller of (excess) and (room). // send what fits
```

```
For each microbial species s living at V: // preserve species mix
```

```
    Compute species share: fraction_s = biomass_s(V) / totalBiomass(V).
```

```
    Move (transfer * fraction_s) of species s from V to N. // proportional split
```

```
Reduce the excess: excess = excess - transfer. // update what is left
```

If excess has reached zero, stop trying neighbors for this voxel. // all placed

If Stage 1 placed all the excess, move on to the next voxel.

Step 1: Identify Excess Voxel & Face Neighbors



Figure B2. Step 1 of the redistribution algorithm: identifying the excess voxel and its face neighbors. Cells show the Manhattan distance to the nearest pore voxel; biomass B (as a fraction of B_{max}) is indicated in square brackets. The purple cell (excess voxel) carries $B = 1.3 B_{max}$, exceeding the allowed maximum by 30%. Its four in-plane face neighbors are shown with their current biomass and available holding capacity. Two additional neighbors in the $\pm z$ directions are also considered in the full 3D algorithm. The combined in-plane holding capacity ($0.22 B_{max}$) is less than the excess ($0.30 B_{max}$), indicating that Stage 1 alone cannot absorb all the surplus.

Step 2 — Stage 1: Capacity-Based Push

Randomly shuffle face neighbors; fill each to B_{max} until excess absorbed or capacity exhausted

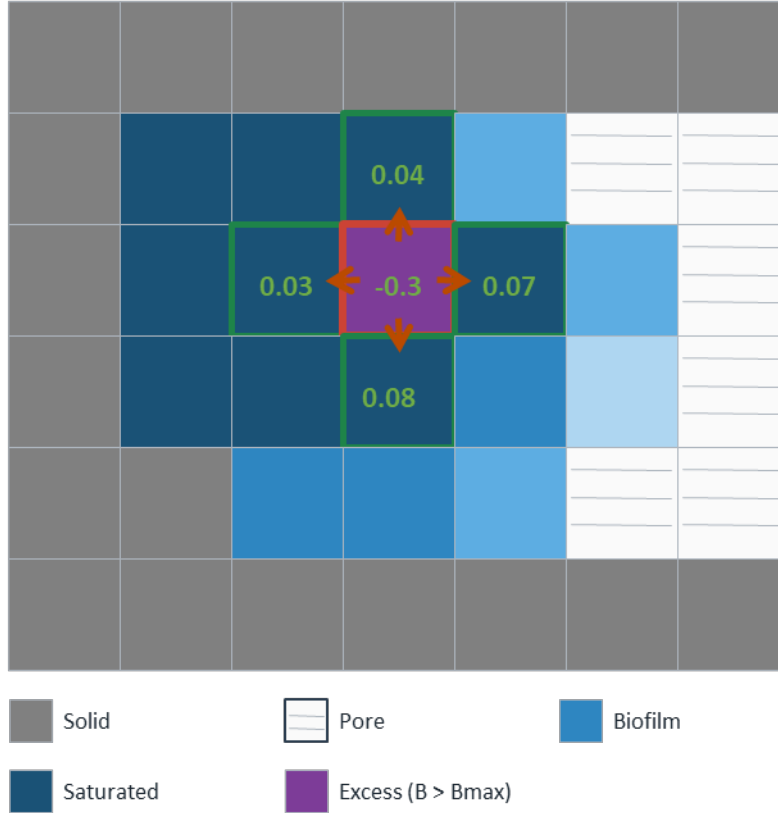


Figure B3. Stage 1 (capacity-based push): the solver checks each face neighbor (green outlines) in a randomly shuffled order and fills each up to B_{max} . Brown arrows indicate the direction of biomass flow from the excess voxel. The available holding capacity ($B_{max} - B$) of each neighbor cell is shown in green. The excess voxel (purple) shows a negative capacity of $-\$0.3 B_{max}$. After Stage 1 has filled all four in-plane neighbors to capacity, the total absorbed amount is $0.22 B_{max}$, leaving a residual of $0.08 B_{max}$ that must be handled by Stage 2.

```
-- Stage 2: push remaining excess toward open pore space --
(Only reached when every neighboring biofilm voxel is already full.)
```

```
Look up the distance-to-nearest-pore value of V.    // precomputed for every biofilm voxel
Among the valid neighbors, find those whose distance-to-pore is
    smaller than that of V.    // they are closer to open space
```

```

If at least one such neighbor exists:
    Pick one of them at random as the target.    // randomize among equally good choices
Otherwise (rare -- deep interior, no neighbor is closer):
    Pick any valid (non-solid) neighbor at random.    // just push somewhere

Push ALL remaining excess to the chosen target.    // target may now be overfilled itself
(Species proportions are preserved, same as Stage 1.)
The outer loop (B.1) will re-handle the target in the next pass.    // biomass migrates step by step

```

— End of Algorithm B.2 —

Figure B4 illustrates Stage 2 in action: the solver compares the Manhattan distance of the excess voxel with those of its neighbors and selects a neighbor whose distance is strictly lower, directing the remaining excess one step closer to the pore boundary. Figure B5 shows the grid after one complete redistribution iteration; the original excess voxel is now at $B_{\max}$, but the chosen target neighbor has inherited a small excess that the next CA pass will continue to push outward.

Step 3 — Stage 2: Distance-Guided Push

Remaining excess pushed toward nearest pore along decreasing Manhattan distance



Figure B4. Stage 2 (distance-guided push): the remaining $0.08 B_{max}$ excess is pushed toward the nearest pore. The solver compares the Manhattan distance of the excess voxel ($dist = 3$) with those of its face neighbors. Only neighbors with a smaller distance will qualify (green checkmarks); neighbors at equal or greater distance are excluded (crosses). One qualifying neighbor will be randomly chosen (yellow outline, $dist = 2$), and all remaining excess is transferred there. The lighter arrows indicate the path the excess will follow toward the pore boundary over successive iterations of the CA loop.

(b) After Redistribution — Single Iteration



Figure B5. State of the grid after one complete redistribution iteration. The original excess voxel (green outline) has been reduced to exactly B_{max} after pushing its surplus to the neighboring cells. All four in-plane neighbors were filled to their maximum capacity during Stage 1. The voxel highlighted in yellow received biomass from both Stage 1 ($0.07 B_{max}$) and Stage 2 ($0.08 B_{max}$), bringing its total to $1.08 B_{max}$ and creating a new excess. The next iteration of the CA loop will redistribute this new excess, transferring the biomass one step closer to the pore boundary.

B.3 Half-Push Redistribution

The full-push mode transfers only the small surplus above the maximum density, which can require many iterations of the outer loop when biomass is concentrated in a few voxels far from open pore space. The half-push mode was

introduced to address this: it uses the same two-stage logic but transfers half of the voxel's total biomass per iteration rather than just the surplus (Figure B6). By moving more biomass per step, the excess reaches open pore space in fewer passes, reducing the number of CA iterations needed for convergence.

— Algorithm B.3: Half-Push Redistribution —

Same two-stage logic as Algorithm B.2, with one change:

```

Instead of: excess = totalBiomass(V) - max_bMassRho    // full-push: only the surplus
Use:       excess = totalBiomass(V) * 0.5              // half-push: half the total biomass

Everything else (Stage 1, Stage 2, species splitting) stays the same.
Moving more biomass per step means fewer passes of the outer loop.    // faster convergence

```

— End of Algorithm B.3 —

Full-Push vs Half-Push Mode

Controlled by `halfflag` parameter: 0 = full-push, 1 = half-push

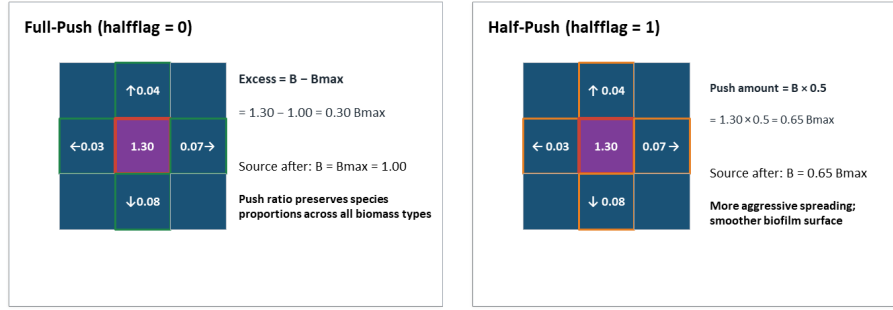


Figure B6. Comparison of the full-push and half-push redistribution modes. Left panel (full-push, `halfflag` = 0): the solver transfers only the surplus above B_{max} , i.e., $excess = B - B_{max} = 0.30 B_{max}$, leaving the source voxel at exactly $B_{max} = 1.00$. Right panel (half-push, `halfflag` = 1): the solver transfers half of the total biomass, i.e., $push\ amount = B \times 0.5 = 0.65 B_{max}$, leaving the source at $0.65 B_{max}$. Both modes use the same two-stage neighbor selection (Stage 1: capacity-based, Stage 2: distance-guided). The half-push redistributes more biomass per step and therefore converges in fewer iterations of the CA loop, at the cost of spreading biomass more aggressively.

B.4 Parallel (MPI) Boundary Exchange

In parallel simulations the 3D domain is decomposed into subdomains, each managed by a separate MPI process. A voxel at the edge of one subdomain may need to push biomass to a neighbor that belongs to a different process. Because each process can only write to its own memory, the transfer is split into two phases (Figure B7, left panel):

1. Push phase (When a push targets a neighbor across a subdomain bound-

ary, the solver writes the biomass amount and a direction tag into a temporary storage lattice on the sending process.

2. Pull phase (After the push step, every process inspects the storage lattice of its neighboring processes at shared boundaries. Stored entries whose direction tag matches are read and added to the local biomass lattice.

This two-phase scheme ensures correct cross-boundary redistribution without direct inter-process memory writes.

B.5 Voxel Reclassification

After each redistribution cycle, the solver updates each voxel's classification using the reclassification threshold (Figure B7, right panel):

$$\text{threshold} = \text{thrd_bFilmFrac} \times \text{max_bMassRho}$$

For every non-solid, non-boundary voxel, the solver sums the biomass of all microbial species at that location and applies two rules:

Biofilm to pore. If a voxel is currently biofilm but its total biomass has dropped below threshold, it is reclassified as pore.

Pore to biofilm. If a voxel is currently pore but its total biomass has reached or exceeded threshold, it is reclassified as biofilm. The biofilm type identifier is set from the microbial species present at that voxel.

— Algorithm B.5: Voxel Reclassification —

Given: biomass densities after redistribution is complete
Goal: update each voxel's identity (pore or biofilm)

Compute the switching threshold:

```
threshold = thrd_bFilmFrac * max_bMassRho    // user-defined fraction of the maximum
```

For each voxel V (skip solids and domain boundaries):

```
Add up all species biomass at V to get bmass.    // total biomass at this location
```

```
If V is currently labeled biofilm but bmass < threshold:    // too little biomass left
```

```
    Change V to pore.    // biofilm has eroded away
```

```
    Copy the pore-type label from the nearest pore neighbor.    // keep pore labels consistent
```

```
If V is currently labeled pore but bmass >= threshold:    // enough biomass accumulated
```

```
    Change V to biofilm.    // new biofilm has grown
```

```
    Set the biofilm-type label from the species present at V.    // label by dominant microbe
```

If any voxel changed identity:

```
    Trigger a recomputation of the steady-state flow field.    // new geometry means new flow
```

— End of Algorithm B.5 —

Any change in the pore-biofilm geometry triggers a recomputation of the steady-state flow field, as described in Section 3.1 of the main text and illustrated in Figure 1.

Voxel Reclassification

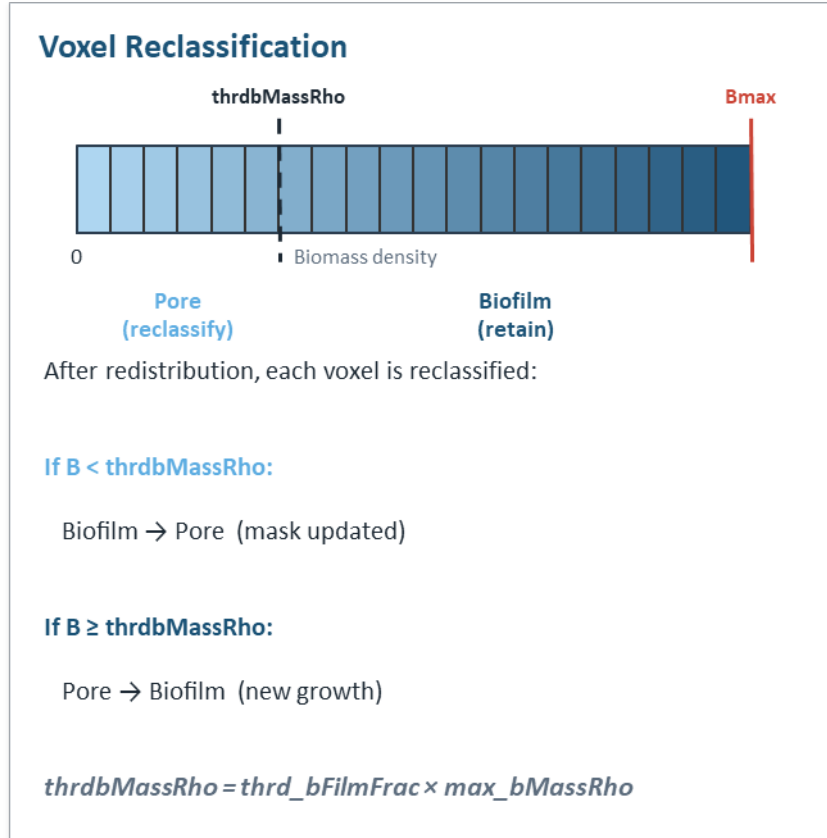


Figure B7. Voxel reclassification threshold. After redistribution completes, each voxel is reclassified based on its total biomass relative to $\text{thrdMassRho} = \text{thrd_bFilmFrac} \times \text{max_bMassRho}$. Voxels which biomass falls below the threshold are reclassified from biofilm to pore; voxels at or above the threshold are reclassified from pore to biofilm. Any reclassification triggers a recomputation of the steady-state flow field.

References:

Jung, H., Song, H.-S., Meile, C.: CompLaB v1. 0: a scalable pore-scale model for flow, biogeochemistry, microbial metabolism, and biofilm dynamics. Geoscientific Model Development. 16, 1683–1696 (2023). doi:10.5194/gmd-16-1683-2023